

| Front-end



Contents

- 01.** React.js 개념과 프로젝트 생성
- 02.** Props & state, ref 의 개념과 Component
- 03.** SPA 앱 구현과 Router
- 04.** Next.js 의 개념과 Routing, 이미지 최적화
- 05.** Network 기능 활용과 로그인 인증 처리
- 06.** 라이브러리 활용과 데이터 공유, 빌드

과정 OVERVIEW

React.js

React.js 개념과 Virtual DOM

React 기본 개념 (state, props, component)

Data 다루기

Todo 리스트 만들어보기

Router

React.js
기본 개념

Data 를 활용한
U.I. 구현

React.js를 이용해
CSR 앱을 구현한다.

과정 OVERVIEW

Next.js

React.js와 Next.js의 차이

Next.js Routing

axios를 활용한 서버와의 통신

Session 처리

프로젝트에서 자주 사용되는 라이브러리

상태 공유와 Context & Redux

Next.js 기초
및 서버 통신

Project 를 위한
실전 개념

Next.js와 외부 라이브러리를
활용한 SSR 앱을 구현한다.

일정(일자별)

React.js 활용부터 Next.js를 활용한 서비스 제작까지

1일차	2일차	3일차
React.js의 개념과 활용	Next.js의 개념과 활용	Next.js의 프로젝트에서 활용
React.js 개념과 Virtual DOM	React.js와 Next.js의 개념 비교	로그인 인증과 JWT 활용
React 기본 개념 이해	Routing 활용	프로젝트에서 사용되는 주요 라이브러리
Data 활용과 앱 만들기	Next.js의 유용한 기능 활용	Prop drilling과 상태 공유의 정의
Router의 개념과 활용	서버와의 통신을 통한 앱 만들기	앱 제작과 배포

강의 목표

Next.js를 활용한 Server와 통신 가능한 서비스 제작

React.js의 개념과 활용

React.js의 기본 개념과 활용 방법에 대해서 알아 본다.

Next.js의 개념과 활용

React.js와 Next.js의 차이점을 확인해 보고 활용 방법에 대해서 알아 본다.

Next.js를 활용한 서비스 제작

Next.js를 활용하여 Server와 통신 가능한 서비스 제작을 해본다.

01.

React.js 개념과 프로젝트 생성

전체 흐름

React.js 의 개념과 활용

React.js
기본개념

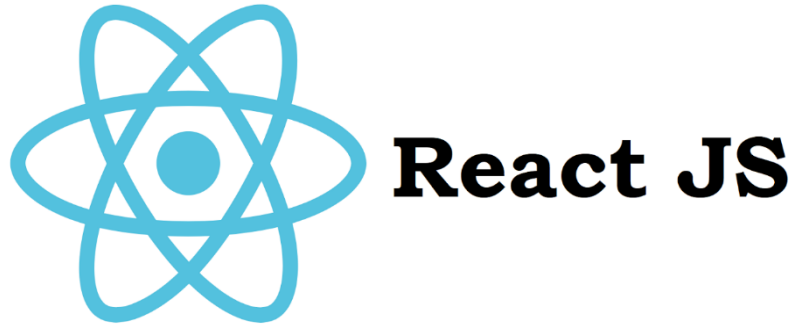
React.js
컴포넌트의
이해

App 작성과
개선을 통한
React.js 활용

React.js 개념

React JS 란?

- React JS는 facebook에서 개발한 View만을 위한 JavaScript Framework
- Component 형태로 각 UI를 구분하며 render() 함수를 사용하여 View를 구현

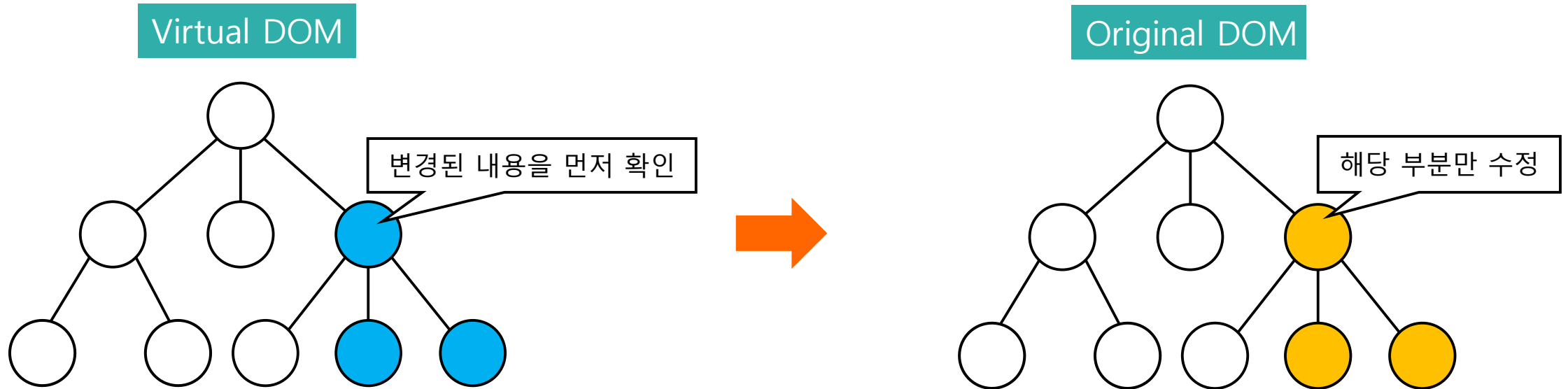


React.js 개념



React JS 란?

- 기존 방식 : DOM 객체에 변화가 생기면 해당 element를 찾아 수정하는 방식
- React : 사본(Virtual DOM)을 통해 수정할 위치를 파악 후 실제 DOM 객체 특정 부분만 변경
- UI의 작은 변화가 있는 곳에 효율적

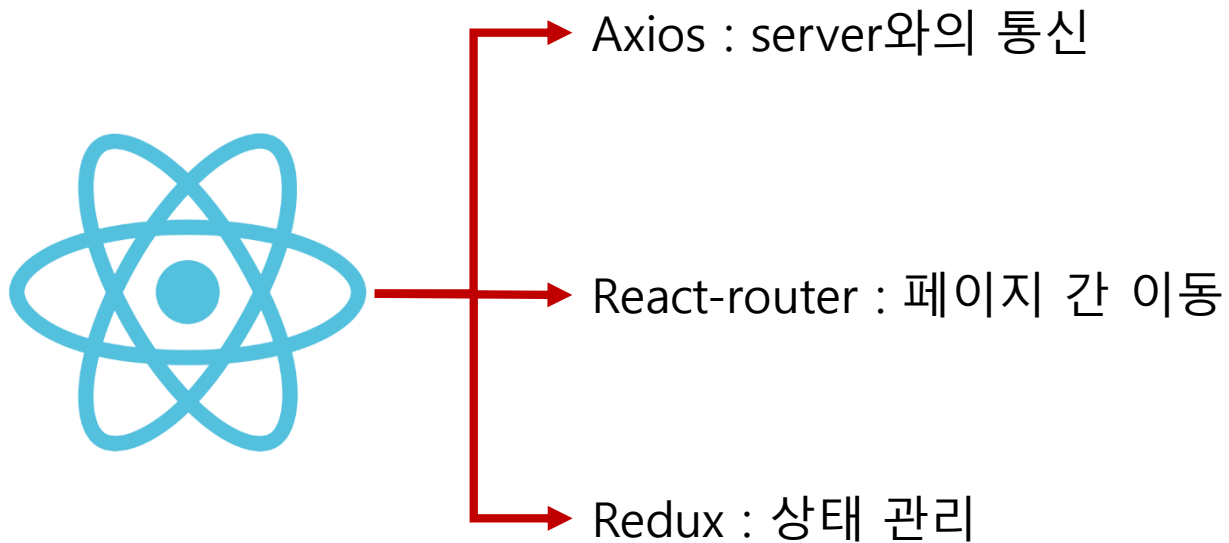


React.js 개념



React JS 란?

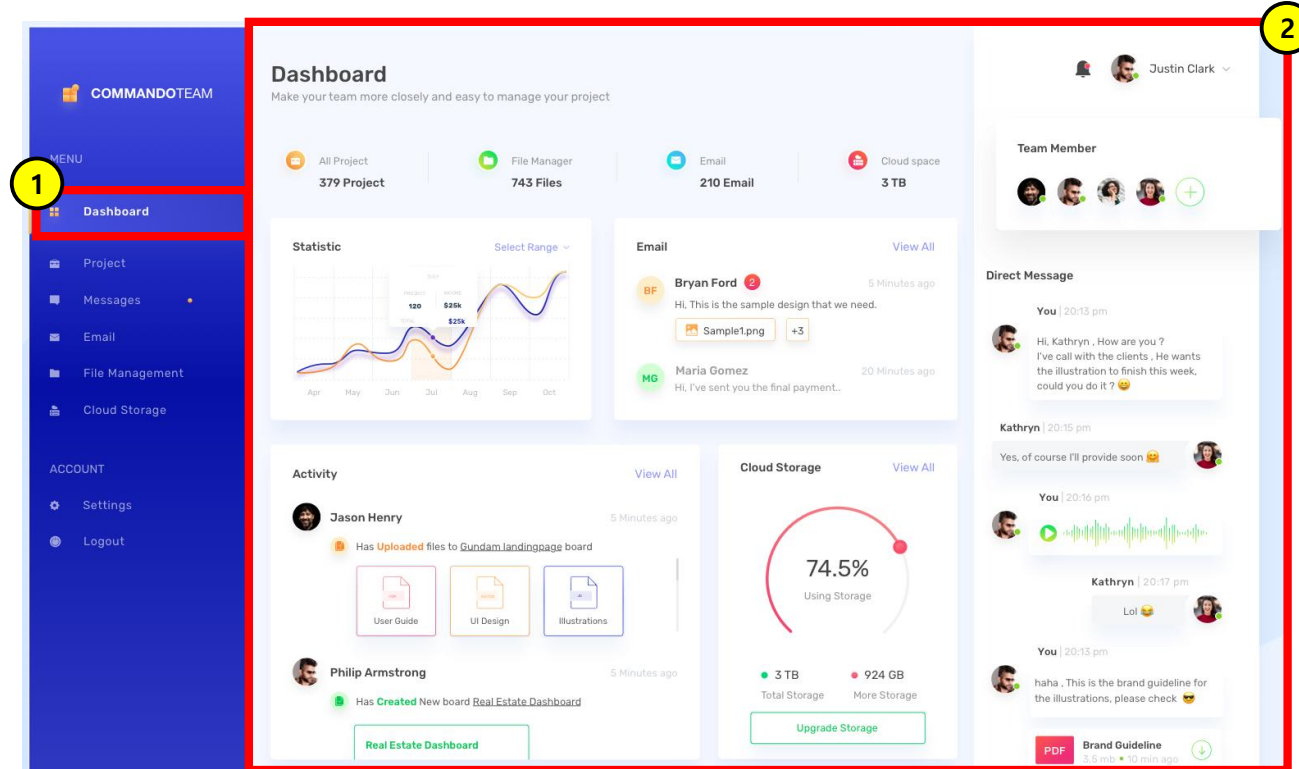
- React는 SPA(Single Page App)의 View만을 담당하는 라이브러리
- 서버 통신, 상태 관리, 페이지 이동 등의 기능은 별도의 라이브러리를 사용해야 함



React.js 개념

☑ React JS 와 SPA(Single Page App)

- SPA(Single Page App)의 개념은 하나의 서비스를 하나의 페이지에서 구현하는 것
- 1번 메뉴를 클릭하면 2번 화면은 요소만 변경되는 개념
- SPA는 Layout 변화는 적지만 그 안에서 실시간적인 변화가 많을 때 적합



React.js 프로젝트

☑ 개발환경 구축

- 먼저 실행과 패키지 관리를 위한 `npm` 사용을 위해 `node.js` 를 설치 해 준다.
- <https://nodejs.org/en/download/prebuilt-installer>

Download Node.js®

Download Node.js the way you want.

Package Manager **Prebuilt Installer** Prebuilt Binaries Source Code

I want the version of Node.js for running

 **Download Node.js v22.12.0**

Node.js includes [npm \(10.9.0\)](#).

Read the [changelog](#) for this version.

Read the [blog post](#) for this version.

Learn how to [verify signed SHASUMS](#)

Check out all available Node.js [download options](#)

Learn about [Node.js Releases](#)

React.js 프로젝트

개발환경 구축

- Node.js 가 설치 되고 나면 cmd 를 실행하여 node.js 가 제대로 설치 되었는지 확인 한다.
- 그리고 react.js App을 만들기 위한 기능을 설치 해 준다.

```
node -v  
npm install -g vite
```

- 원하는 폴더에서 **npm create vite**
- 이름은 **5자 이상**에 **대문자**가 있어서는 안된다.(예:myapplication) / _바 사용 가능
- 아래 내용 설정
프로젝트 명(first_project)
framework(React)
사용 언어(JavaScript)
Linter(ESLint)
Install with npm and start now? **No**

React.js 프로젝트

개발환경 구축

- 그리고 `react` 의 기본 패키지를 설치해 주자!
- 생성된 폴더에 들어가서 `npm` 을 명령어를 설치해 주고...

```
cd start_app  
npm install
```

- 개발용 서버를 실행해 준다.

```
npm run dev
```

React.js 프로젝트

☑ 개발환경 구축

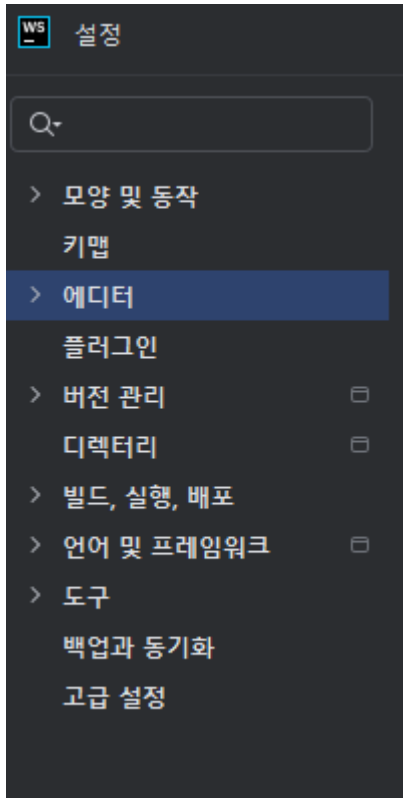
- React 를 작업할 IDE 를 설치 해 준다.
- <https://www.jetbrains.com/ko-kr/webstorm/>



React.js 프로젝트

☑ 개발환경 구축

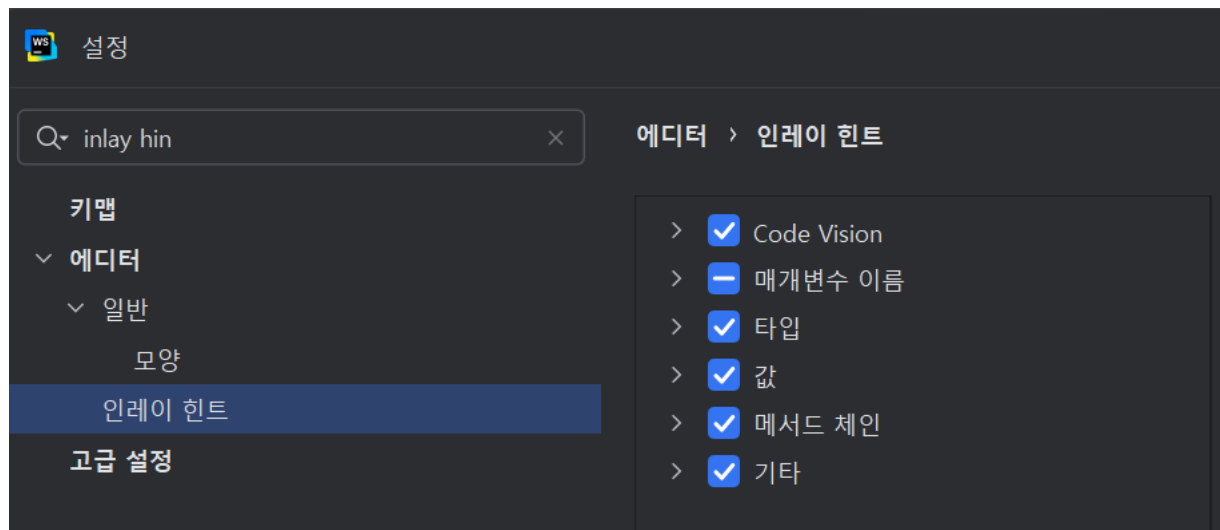
- 메뉴 > 설정 > 키 맵 에서 Eclipse 로 지정해 주자
- 메뉴 > 설정 > 글꼴 에서 글자 크기와 글꼴을 지정해 주자



React.js 프로젝트

☑ 개발환경 구축

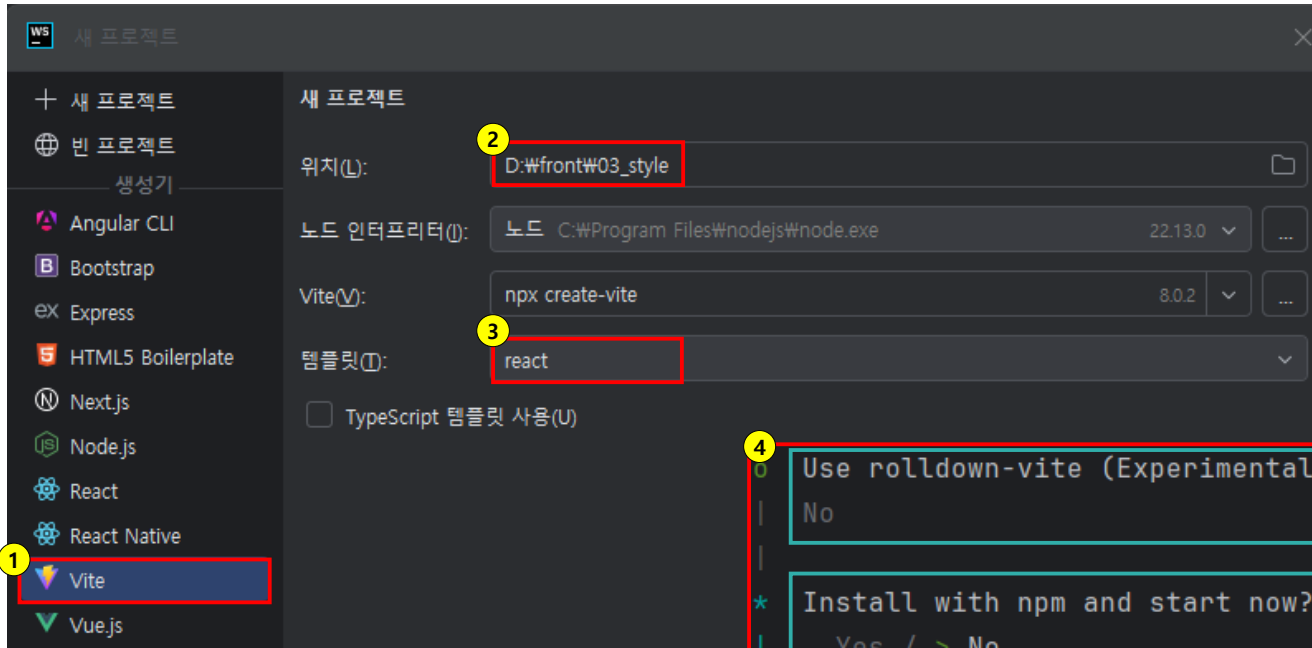
- 메뉴 > 설정 > inlay hint 검색
- 타입 부분 체크를 지워주면 시인성을 개선할 수 있다.



React.js 프로젝트

☑ React JS 프로젝트 생성

- 실습을 통해 프로젝트를 만들어보고 구조를 살펴보자
- Alt+₩ > File > New > Project... 선택 후 아래 내용대로 프로젝트를 생성해 보자



1. Vite 선택 (개발 및 빌드 도구)
2. Project 생성 경로와 이름 지정 (5자 이상, 대문자X)
3. 템플릿 react 선택
4. 설정

→ 최신 번들러 사용 여부(No)

→ npm install 과 서버 당장 실행 여부 (Yes|No)

실습파일 : 01_start

React.js 프로젝트

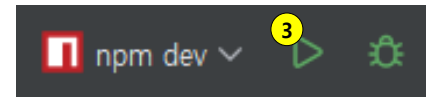
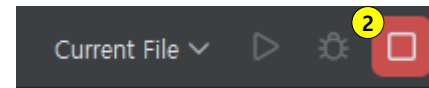
React JS 프로젝트 생성

- Install with npm and start now? > Yes 를 선택한 경우
 1. 바로 서버가 실행
 2. 상단의 정지 버튼을 누르면 서버가 정지하면서 소스가 생성
 3. 이후 실행 버튼 활성화

1

```
VITE v7.1.11 ready in 285 ms

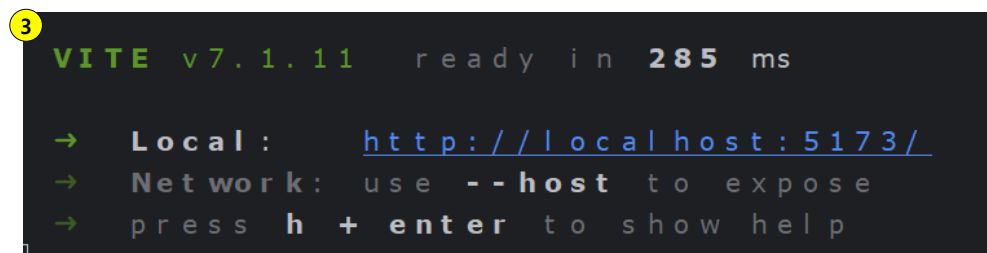
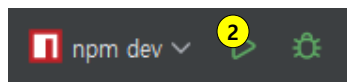
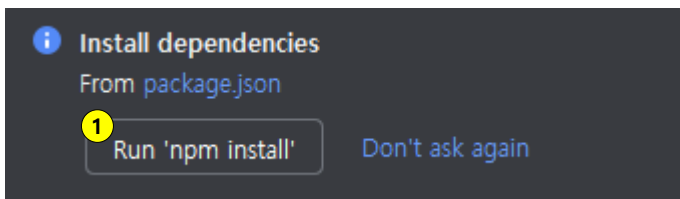
→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```



React.js 프로젝트

React JS 프로젝트 생성

- Install with npm and start now? > No 를 선택한 경우
 1. 우측 하단의 Run 'npm install' 버튼을 눌러 의존성을 설치
 2. 상단의 녹색 화살표 버튼을 눌러준다.
 3. 서버 실행 모습



React.js 프로젝트



React JS 프로젝트 생성

- npm 설치 관련 오류가 발생한다면?

! 'vite'은(는) 내부 또는 외부 명령, 실행할 수 있는 프로그램, 또는 배치 파일이 아닙니다.

1. Terminal을 열고, 현재 프로젝트 경로와 일치하는지 확인하세요.
2. `npm install vite` 명령어를 입력하세요.
3. `npm run dev` 명령어를 입력하여 vite를 재실행하세요.

```
Terminal Local x
Windows PowerShell
Copyright (C) Microsoft Corporation.

새로운 기능 및 개선 사항에 대한 최신 PowerShell

PS D:\front\create_project
```

1

```
Terminal Local x
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

새로운 기능 및 개선 사항에 대한 최신 PowerShell

PS D:\front\create_project npm install vite

up to date, audited 158 packages in 1s

33 packages are looking for funding
run `npm fund` for details
```

2

```
Terminal Local x
found 0 vulnerabilities
PS D:\front\create_project

> checking@0.0.0 dev npm run dev
> vite

VITE v7.2.2 ready in 575 ms

→ Local: http://localhost:5173/
→ Network:
→ press h + enter to show help
```

3

서버주소

React.js 프로젝트



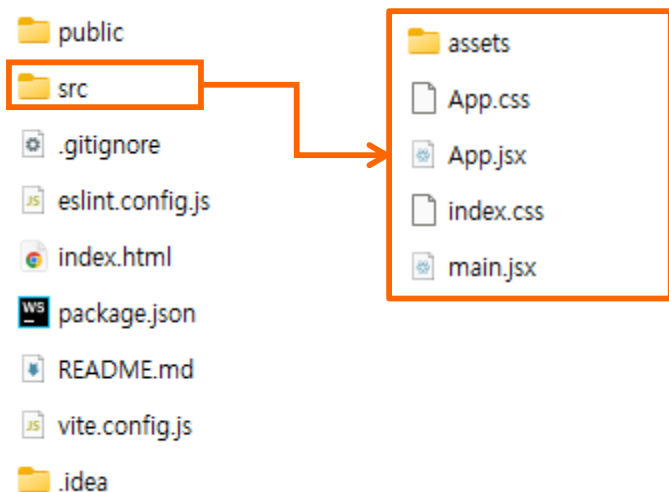
- Node Package Manager는 node.js에서 사용하는 라이브러리를 관리해주는 도구
- npm 명령만으로 원하는 라이브러리 설치·제거가 가능
- React.js 개발 시에도 npm을 통해 원하는 라이브러리를 설치할 예정

명령어	설명
npm install 설치라이브러리	지정한 라이브러리 설치
npm uninstall 설치된라이브러리	지정한 라이브러리 삭제
npm update	설치된 라이브러리 업데이트
npm run dev build lint preview	package.json 파일에 있는 script 명령 실행 <pre>"scripts": { "dev": "vite", "build": "vite build", "lint": "eslint .", "preview": "vite preview" }</pre>

React.js 프로젝트

☑ React JS 프로젝트 구조

- src 폴더에는 실제 코드들이 존재
- App.js 파일은 화면에 표현될 내용들에 대한 code가 존재
- assets에는 image 등의 resource들을 보관
- node_modules 폴더에는 설치한 component들이 존재



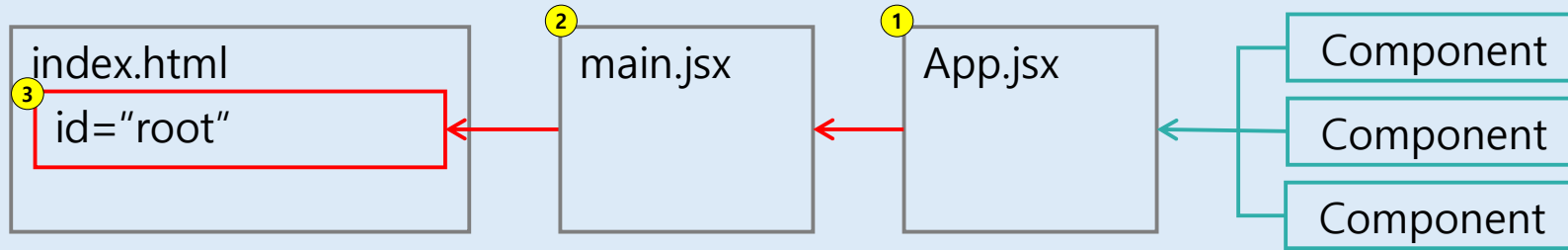
React.js 프로젝트



React JS 프로젝트 구조

- App.jsx에서 필요한 Component를 조합하여 main.jsx 통해 구현

① 여러 컴포넌트로 구성된 App.jsx를 ② main.jsx 에서 호출하고, ③ index.html 페이지에 그려서 표출하는 구조



```
<div id="root"></div>
<script type="module" src="/src/main.jsx"></script>
```

index.html

```
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

실습 파일 : 01_start > src > main.jsx

```
const html = (<div>Hello, React.js</div>);
function App() {
  return (
    <div>{html}</div>
  );
}
export default App
```

실습 파일 : 01_start > src > App.jsx

JSX



- JSX는 JavaScript를 확장한 문법
- JSX를 활용하면 HTML rendering 시 JavaScript 문법을 사용할 수 있게 됨

JSX is a **syntax extension for JavaScript** that lets you write HTML-like markup inside a JavaScript file.

출처 : <https://react.dev/learn/writing-markup-with-jsx>

JSX

☑ JSX 활용법을 익혀 보자

```
export default function App(){  
  let name = '김지훈';  
  const gender = '남자';  
  let age = 30;  
  ...  
}
```

• 결과 화면

안녕하세요 리액트에 잘 오셨어요

김지훈/남자/30

맞네요

맞네요

서른

```
export default function App(){  
  ...  
  return(  
    <>  
      <div className="App"><h3>안녕하세요 리액트에 잘 오셨어요  
    </h3></div>  
      <div>{name}/{gender}/{age}<br/></div>  
      <div>  
        {  
          /* eslint-disable-next-line no-constant-condition */  
          {1+1 === 2 ? (<p>맞네요</p>):( <p>틀리네요</p>)}  
        }  
        (( )=>{  
          if(age === 11) return (<div>열</div>);  
          if(age === 30) return (<div>서른</div>);  
        })()  
      </div>  
    </>  
  );  
}
```

항상 참 또는 거짓인 조건식을 의도적으로 허용하기 위해 아래 내용이 추가되어야 한다.

```
{/* eslint-disable-next-line no-constant-condition */}  
{1+1 === 2 ? (<p>맞네요</p>):( <p>틀리네요</p>)}  
{  
  (( )=>{  
    if(age === 11) return (<div>열</div>);  
    if(age === 30) return (<div>서른</div>);  
  })()  
}
```

조건문 활용

실습파일 : 02_jsx > src > App.jsx

Style

✓ Style

- React.js 에서도 기존과 같이 HTML 과 CSS 를 사용 가능
- css 파일과 js 파일에서의 **사용법이 약간의 차이**가 존재
- 아래의 **동일한 스타일 적용 예시**를 보고 CSS 방식과 JS 방식의 차이를 확인해 보자

CSS

```
span{  
  font-size: 26px;  
  font-weight: 700;  
  color: blue;  
  background-color: yellowgreen;  
}
```

- 하이픈(-) 사용
- 대부분 단위를 명시해야 함

```
<div>  
  <span>Use Style Sheet</span>  
</div>
```

=

JS

```
const styles = {  
  fontSize: 26,  
  fontWeight: 700,  
  color: 'blue',  
  backgroundColor: 'yellowgreen'  
};
```

- 카멜 표기법 사용
- 단위가 없다면 px로 처리됨

```
<div>  
  <span style={styles}>Use Style Sheet</span>  
</div>
```

Style 실습

☑ JSX에 스타일을 추가해 보자!

```
span{  
  color: dodgerblue;  
  font-size: 26px;  
  font-weight: 700;  
}
```

실습파일 : 03_styles > src > App.css

```
const styles = {  
  fontSize: 26,  
  fontWeight: 700,  
  color: 'blue',  
  backgroundColor: 'yellowgreen'  
};
```

```
<div>  
  <h1 style={{color:'red'}}>Hello Inline style</h1>  
  { /*직접 스타일 부여*/}  
  
  <p style={styles}>JavaScript Object Style</p>  
  { /*자바스크립트 객체에 스타일 부여*/}  
  
  <span>Use Style Sheet</span>  
  { /*css 로 특정 태그에 부여*/}  
  
  <h1 className="App-title">Use ClassName</h1>  
  { /*클래스 이름으로 부여*/}  
</div>
```

실습파일 : 03_styles > src > component > MyStyle.jsx

- 결과 화면

Hello Inline style

JavaScript Object Style

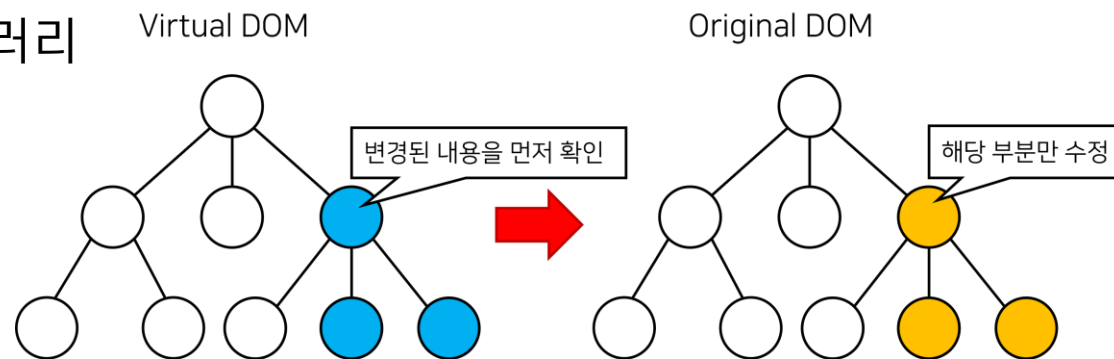
Use Style Sheet

Use ClassName

Summary

☑ React JS 개념

- Virtual DOM을 통해 수정할 위치를 파악 후 실제 DOM 객체 특정 부분만 변경하는 개념
- UI 의 작은 변화가 있는 곳에 효율적
- SPA(Single Page App) 의 View 만을 담당하는 라이브러리



☑ React JS 프로젝트

- App.jsx에서 필요한 Component를 조합하여 main.jsx 통해 구현
- 이때 JSX는 JavaScript를 확장(extends)한 문법
- jsx를 통해 style 다룰 수 있음

02.

Props & State, ref의 개념과 Component

Props와 State

React의 개념들

- React.js에서 Props와 State는 컴포넌트가 데이터를 관리하고 상호작용하는 데 사용되는 핵심 메커니즘
- props와 state가 바뀔 때마다, React가 화면을 다시 그림 (= render()가 실행된다.)
- props와 state의 차이점을 살펴보자

	Props	State
생성방법	부모 component에서 전달받는다	setState() 를 통해 생성한다.
수정여부	수정/추가 불가능	setState()를 이용해 수정/추가 가능
Render 호출 시 점	변경 감지 시	변경 감지 시

Props와 State

☑ React의 개념들

- Props 의 값 생성 및 활용 예시

부모 Component

자식(PropBtn)을 호출하면서 name이라는 props 전달

```
<PropBtn name="This is Prop button"/>
```

자식 Component

부모 component가 넘겨준 props를 받아서

{name}으로 출력

```
const PropBtn = ({name}) => {  
  return(  
    <div style={{margin:10}}>{name}</div>  
  );  
}
```

- state 의 값 생성 및 활용 예시

useState() 를 통해 state 를 생성하고 초기값 설정

```
const StateBtn=() => {
```

```
  const[count,setCount] = useState(100);
```

```
  let updateCount = () => {
```

```
    setCount(count-1);
```

```
  }
```

```
  return(  
    <div style={{margin: 10}}>
```

```
      <button onClick={()=>{updateCount()}}>
```

```
        down count : {count}
```

```
      </button>
```

```
    </div>
```

```
  );
```

```
}
```

특정한 상황시 setCount() 함수를 통해 값 변경

Props와 State

☑ state 와 UI 변화의 연관성을 확인해 보자

```
<PropBtn name="This is Prop button"/>
```

실습파일 : 04_props_state > src > App.jsx

//Props에는 상위 컴포넌트에서 전달받은 값이 담겨있으며 변경 불가능하다.

const PropBtn = ({name})=>{//App 에서 전달 해준 name 값을 {name} 형태로 받을 수 있다.

```
const sendMsg = (name) => {  
  alert(`Your name is ${name}`);  
}
```

```
return(  
  <div style={{margin:10}}>  
    <button  
      onClick={()=>{sendMsg(name)}}>  
      {name}  
    </button>  
  </div>  
);
```

```
export default PropBtn;
```

부모 component로부터 받아온 props 값은 UI rendering에 적용

실습파일 : 04_props_state > src > component > PropBtn.jsx

Props와 State

☑ state 와 UI 변화의 연관성을 확인해 보자

```
const StateBtn=() => {  
  
  const[count,setCount] = useState(100); //count 변수와 setCount 함수를 지정하여 초기값을 100으로 지정  
  
  let updateCount = () => {  
    setCount(count-1); // setCount를 사용해 새로운 값 수정  
  }  
  
  return(  
    <div style={{margin: 10}}>  
      <button onClick={()=>{updateCount()}}>  
        down count : {count}  
      </button>  
    </div>  
  );  
}  
export default StateBtn;
```

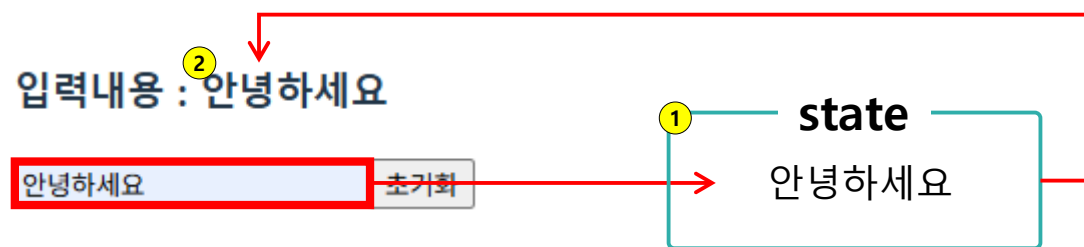
생성된 state 값은 UI rendering에 적용

실습파일 : 04_props_state > src > component > StateBtn.jsx

Props와 State

input 태그 다루기

- UI의 목적은 서버로 특정 내용을 보내거나 받기 위한 것
- UI에 있어서 input은 아주 기본적인 기능
- input에서 입력받은 값을 변경할 때도 역시 state가 사용됨



```
const [text, setText] = useState('');

function getText(e){
  ① setText(e.target.value);
}

return(
  <div>
    <h3>입력내용 : ② {text}</h3>
  </div>
);
```

실습파일 : 05_inputs > src > component > Input.jsx

Props와 State

☑ 태그를 다뤄보자

```
const [inputs, setInputs] = useState({nick:'',name:''});  
const {name, nick} = inputs;  
  
const typing=(key,e) => {  
  let val = e.target.value;  
  setInputs({ //setInput 함수를 활용하여 inputs 를 복사 후 키:값 형태로 추가  
    ...inputs  
    ,[key]:val  
  });  
}
```

실습파일 : 05_inputs > src > component > Inputs.jsx

• 결과 화면

입력내용 :

아이디:

닉네임:

아이디 : aivleschool / 닉네임:에이블스쿨

Props와 State

☑ 비구조 할당 활용

- 객체에서 특정 값을 변수에 담는데 비구조 할당 사용 시 편리함

```
const {name, nick} = inputs; // 이렇게 하나의 객체에서 복수개의 값을 받아낼 수도 있다.
```

- 아래와 같이 객체 내 특정 속성들을 호출 시 비구조 할당을 사용하면 더욱 간결하게 가져올 수 있음
예) 10개의 속성을 가져올 경우 일반적 방식으로는 10줄의 코드가 생성되지만 비구조 할당은 한 줄로 처리 가능

일반적인 프로퍼티 접근

```
const obj = {name:'kim', nick:'zer0'};  
  
let name = obj.name;  
let nick = obj.nick;
```

V.S.

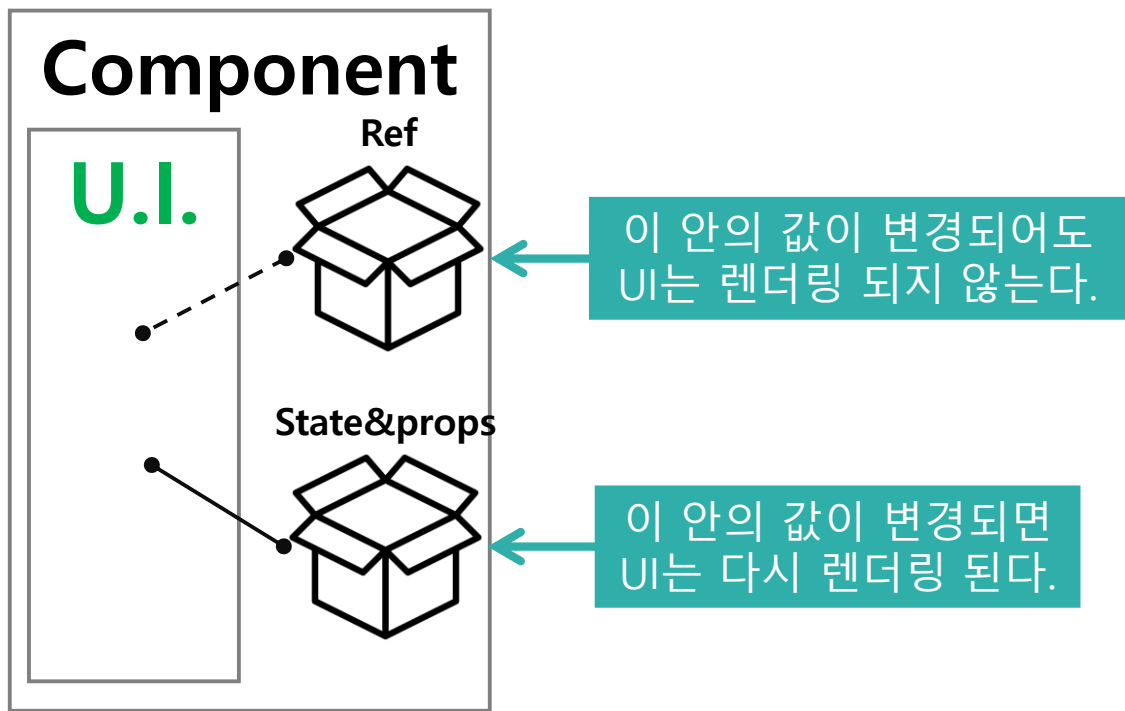
비구조할당

```
const obj = {name:'kim', nick:'zer0'};  
  
let {name, nick} = obj;
```

Ref 객체

☑ Ref 객체

- Ref는 State나 Props처럼 **값을 담는 객체**
- 하지만 State나 Props와 다르게 **렌더링 시 영향 받지 않음**
- 그래서 DOM에 직접 접근이 필요하거나 렌더링에 영향 받지 않는 값을 저장할 때 사용



• DOM 직접 접근 시 렌더링 되면 안되나요?

- 렌더링은 변경된 state 값으로 Virtual DOM을 만들고 실제 DOM에 적용하는 과정이다.
- 이때 실제 DOM에 직접 접근하면서 렌더링 까지 일어난다면 여러 문제가 발생 할 수 있다.

Ref 객체

✓ Ref 객체

- useRef() 를 통해 객체 생성, current 속성에 값을 저장하거나 가져옴
- DOM 에 직접 접근을 위해서는 해당 요소에 ref 속성을 추가해야 함

```
let inputRef = useRef();
let [val, setVal] = useState('');
const txtFocus=() => {
  console.log([inputRef.current]); /* 버튼을 누르면(1) input 에 있는
  inputRef.current.focus();       값이
  setVal('');                     console 로 출력 된다.*/
};
return(
  <div>
    <h3>입력값:{val}</h3>
    <input type="text" value={val} onChange={chgText} ref={inputRef}/>
    <button onClick={txtFocus}>TAB</button>
  </div>
);
```

입력값:hello React.js

hello React.js

TAB

실습파일 : 06_ref > src > component > GetElem.jsx

Ref 객체

☑ ref 객체를 변수로 이용하는 방법에 대해서도 알아보자

```
const [stateVal, setStateVal] = React.useState(0);
const refVal = React.useRef(0);
function updataState(){
  setStateVal(stateVal+1); /* 버튼(1) 을 누를 때마다 값이 update 된다.*/
}
function updateRef(){
  refVal.current += 1; /* 버튼(2) 을 눌러도 값이 update 되지 않는다.*/
}
return(
  <div>
    <button onClick={updataState}>
      state count : {stateVal}</button>
    <button onClick={updateRef}>
      ref count : {refVal.current}</button>
    </div>
  );
```



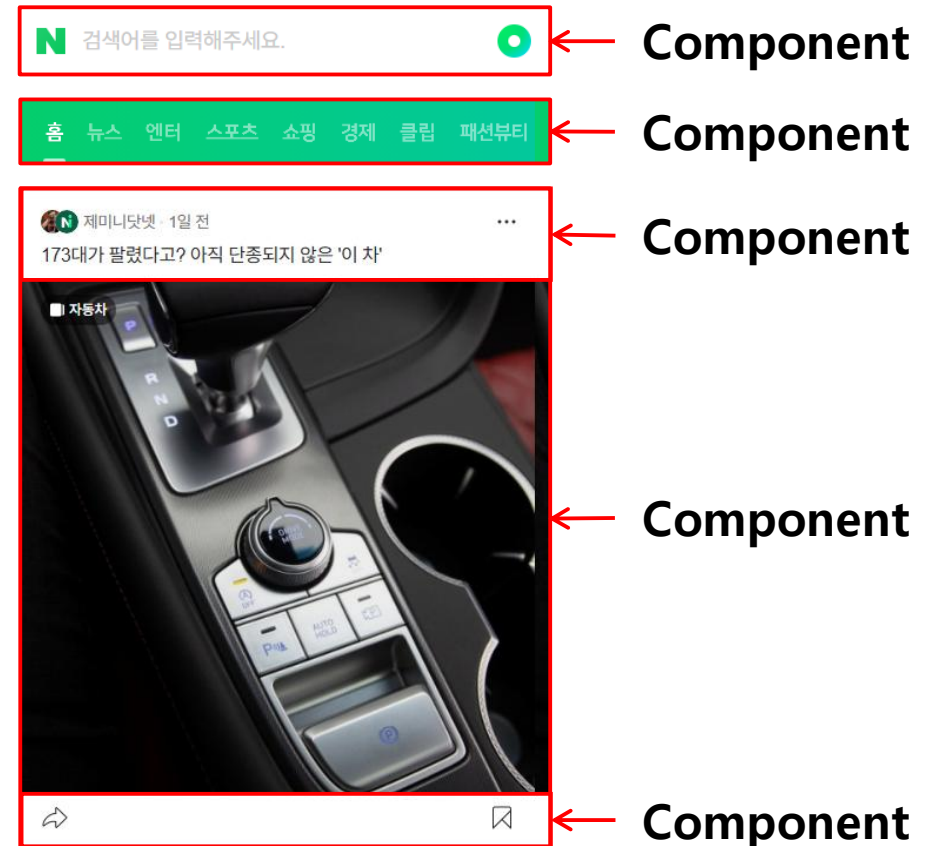
실습파일 : 06_ref > src > component > RefVar.jsx

Component

☑ Component

- 하나의 UI 는 Component 의 구성이라고 볼 수 있음
- 예시처럼 하나의 page를 여러 Component를 조합하여 구성

- Component 들로 구성된 페이지 예시



Component

☑ Component

- component의 render() 함수의 영향에 대해 알아보자
- 아래의 현상에 대한 이유를 알아보자

1. ①클릭을 3번 한 후
2. ②SHOW ALERT 을 누르고
3. 그동안 ①Click me 를 열심히 눌러보자!!
4. alert 에는 3번만 클릭한 것으로 나타난다.

실습파일 : 07_comp > src > Comp.jsx



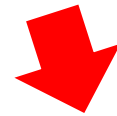
You Click 14 times

①

Click me

②

Show Alert me



③

localhost:19006 내용:

You clicked ON : 3

확인

Component

Component

```
export default function Comp(){
  const [count, setCount] = React.useState(0);
  const refVal = React.useRef(0);
  ...
  function updateCount(){
    setCount(count+1); //state는 랜더링 상황에서만 영향을 준다.
    refVal.current = count; //ref는 render와 관계없이 값을 변화 시킨다.
  }
  const alertCount=() => {
    setTimeout(() => {
      alert("You clicked ON : "+count); // alert이 되는 순간 값을 업데이트하지 못한다.
      /* alert 이후에도 값이 update 되도록 코드를 작성해 보자 */
    }, 3000);
  }
  return ( ... );
}
```

실습파일 : 07_comp > src > Comp.jsx

Component

✓ Component Life Cycle

- Component는 개체처럼 자체의 생애 주기가 있음
- 생성부터 소멸까지 특정 시점에 작용하는 기능을 `useEffect()`를 통해 구현 가능

사용법	설명
<code>useEffect(()=>{ ... });</code>	컴포넌트가 렌더링하면 무조건 ... 수행
<code>useEffect(()=>{ ... },[]);</code>	컴포넌트가 최초 렌더링 될 때 ... 수행

• 사용 예시

```
// 렌더링하면 무조건 호출 된다.  
// 어떤 버튼을 눌러도 반응 한다.  
useEffect(() => {  
  console.log('렌더링되면 호출');  
});  
  
// 2. 컴포넌트가 최초 렌더링될 때 호출  
// 버튼을 눌러도 호출되지 않는다.  
useEffect(()=>{  
  console.log('컴포넌트가 최초 렌더링 될때 호출');  
},[]);
```

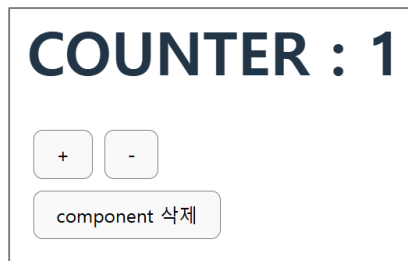
실습파일 : 08_lc_api > src > Comp.jsx

Component

✓ Component Life Cycle

사용법	설명
<code>useEffect(()=>{ ... },[number]);</code>	컴포넌트의 number라는 state에 변화가 생기면 ... 수행
<code>useEffect(()=>{ ... return ()=>{ *** } },[]);</code>	최초 컴포넌트 등장 시 ... 수행 컴포넌트 삭제 전 *** 수행

• 결과 화면



• console

컴포넌트가 최초 랜더링 될때 호출
number state 변화에 호출
최초 호출
number state 변화에 호출
컴포넌트 제거직전

• 사용 예시

```
//3. 특정 state 변화에 호출
//number가 변경되어 컴포넌트가 재렌더링된 이후 실행
useEffect(()=>{
  console.log('number state 변화에 호출');
},[number]);

//4. 컴포넌트가 수명을 다하고 사라질때
useEffect(()=>{
  console.log('최초 호출');
  return ()=>{
    //0.이전 컴포넌트가 사라질때 실행
    console.log('컴포넌트 제거직전');
  }
},[]);
```

실습파일 : 08_lc_api > src > Comp.jsx

Summary

✓ Props와 State

	props	state
특징	부모 component에서 전달받은 값 사용	setState()에서 전달한 값 사용
변경 가능 여부	변경이 불가능	setState()를 이용해 변경 가능
render() 실행 여부	변경이 감지될 때 render()가 실행	변경이 감지될 때 render()가 실행

✓ Ref 객체

- 특정한 DOM 요소를 담을 때 Ref를 활용할 수 있음
- ref는 렌더링에 영향 받지 않는 변수로 활용 가능

```
let inputRef = useRef();  
return(  
  <div>  
    <input type="text" ref={inputRef}/>  
  </div>  
)
```

```
const refVal = React.useRef(0);  
return(  
  <div>  
    ref count : {refVal.current}</button>  
  </div>  
)
```

Summary

Component

- 하나의 UI는 Component의 구성이라고 볼 수 있음
- component 생성부터 소멸까지 특정 시점에 작용하는 기능을 `useEffect()`를 통해 구현 가능

사용법	설명
<code>useEffect(()=>{ ... });</code>	컴포넌트가 렌더링 하면 무조건 ... 수행
<code>useEffect(()=>{ ... },[]);</code>	컴포넌트가 최초 렌더링 될 때 ... 수행
<code>useEffect(()=>{ ... },[number]);</code>	컴포넌트의 number라는 state에 변화가 생기면 ... 수행
<code>useEffect(()=>{ ... return ()=>{ *** } },[]);</code>	최초 컴포넌트 등장 시 ... 수행 컴포넌트삭제 전 *** 수행

03.

SPA 앱 구현과 Router

SPA 만들기

React.js로 SPA 만들기

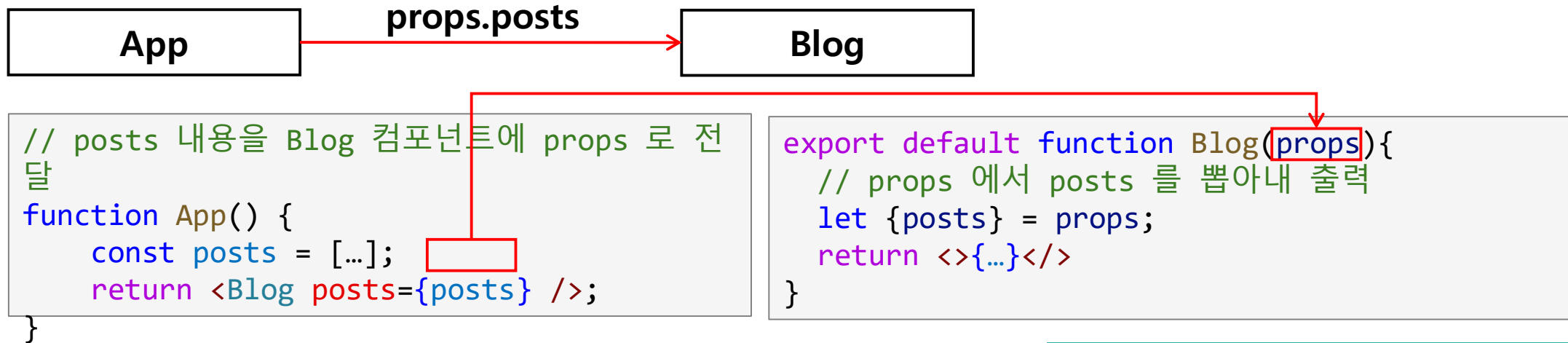
- JavaScript에서는 복수 개의 데이터를 넣기 위한 방법으로 Array와 Object를 사용
- React.js App 에서도 기본적인 복수 데이터의 저장의 형태는 Array와 Object
- 이를 조합한 형태가 JSON(JavaScript Object Notation)

```
const posts = [  
  {id:1, title:'박찬욱 감독님 표 기생충 -어쩔수가없다 관람후기', category:'영화'},  
  {id:2, title:'광명 철산역 숨은맛집 소들넉, 소고기 회식 추천 고기집', category:'맛집'},  
  {id:3, title:'컴포즈커피 반값할인 50% 오키클럽행사', category:'카페'},  
  {id:4, title:'엘릭서랩(Elixir Lab) 신제품 엑소 크림 4종 전격 리뷰! PDRN 라인까지', category:'미용'},  
  {id:5, title:'동급 최강! 가성비 모델 랜드로버 디스커버리 스포츠 시승기', category:'자동차'},  
  {id:6, title:'오이도 명물 등대빵 노을바다 뷰 즐기기 좋은 카페 베이커리 방문기', category:'카페'},  
];
```

SPA 만들기

✓ Array와 Object를 표출하는 앱을 만들어 보자

- 작성할 앱의 Component 구조를 참고하자



실습파일 : 09_list > src > App.jsx

SPA 만들기

```
export default function Blog(props){
```

부모 컴포넌트로부터 받은 props 객체

```
  let {posts} = props;
```

Props로부터 비구조 할당으로 필요한 속성값 할당

```
  const style = {  
    borderBottom: '1px solid lightgray',  
  }
```

적용할 style 속성을 담고 있는 객체

```
  const list = posts.map((post) => {  
    return (<div style={style} key={post.id}>  
      <h3>{post.id} : {post.title}</h3>  
      <p>category : {post.category}</p>  
    </div>);  
  })
```

Posts로부터 데이터를 하나씩 뽑아 jsx
로 만들고 list변수에 할당

```
  return <>{list}</>
```

list 변수에 담긴 jsx 출력

```
}
```

실습파일 : 09_list > src > Blog.jsx

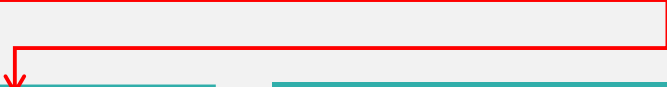
SPA 만들기

```
import Blog from "../Blog.jsx";

function App() {
  const posts = [
    {id:1, title:'박찬욱 감독님 표 기생충 -어쩔수가없다 관람후기', category:'영화'},
    {id:2, title:'광명 철산역 숨은맛집 소들넉, 소고기 회식 추천 고기집', category:'맛집'},
    {id:3, title:'컴포즈커피 반값할인 50% 오키클럽행사', category:'카페'},
    {id:4, title:'엘릭서랩(Elixir Lab) 신제품 엑소 크림 4종 전격 리뷰! PDRN 라인까지', category:'미용'},
    {id:5, title:'동급 최강! 가성비 모델 랜드로버 디스커버리 스포츠 시승기', category:'자동차'},
    {id:6, title:'오이도 명물 등대빵 노을바다 뷰 즐기기 좋은 카페 베이커리 방문기', category:'카페'},
  ];

  return <Blog posts={posts} />;
}

export default App;
```



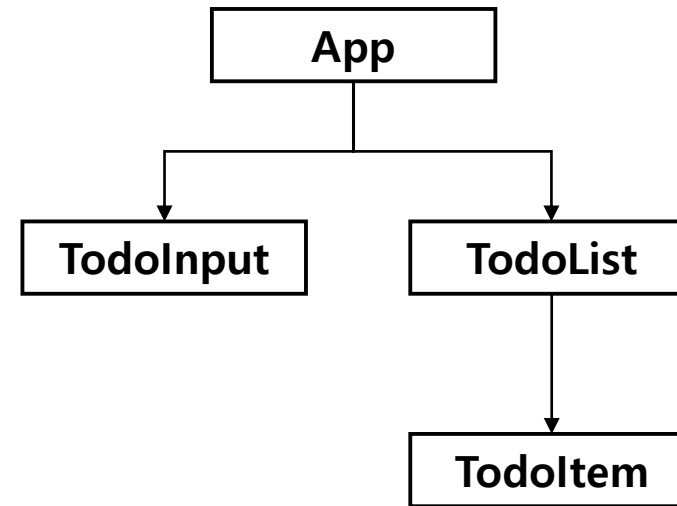
posts 변수에 할당된 데이터를 Blog 컴포넌트에 posts 라는 이름으로 전달

실습파일 : 09_list > src > App.jsx

SPA 만들기(심화)

☑ 할 일을 추가하고, 한 일 처리와 삭제 기능이 있는 App을 만들어 보자

- 아래 UI와 Component 구조를 확인해 보자



SPA 만들기(심화)

☑ 할 일을 추가하고, 한 일 처리와 삭제 기능이 있는 App을 만들어 보자

- 다음 결과 화면과 스켈레톤 코드의 주석을 보고 앱의 기능을 완성해 보자

```
export default function App() {  
  const [input, setInput] = useState('');  
  const [list, setList] = useState([]);  
  ...(생략)...  
  
  // 할일 추가  
  const onInsertHandler=() => {  
    // id 증가 -> id, text, done을 state 에 추가  
  };  
  
  // 할일 처리(체크, 취소선 처리)  
  const onToggleHandler=(id) => {  
    // list 에서 클릭한 id 값과 동일한 값이 있는 인덱스를 찾아서  
    // state 값을 변경한다.  
    //그리고 변경된 값을 저장  
  };  
  
  ...(생략)...  
}
```

- 결과 화면

해야 할 일

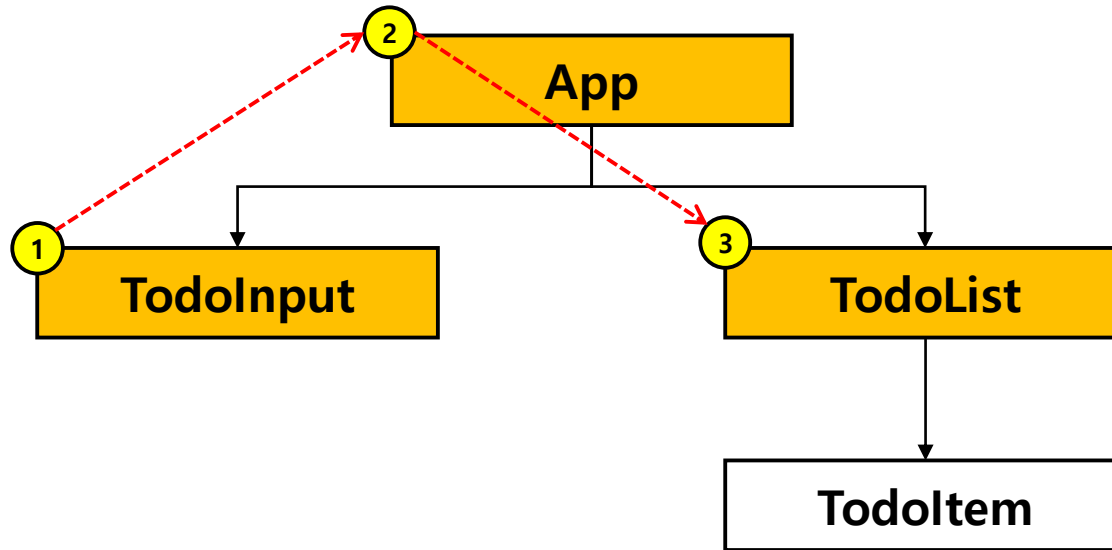
	추가
☒ Spring 공부하기	[삭제]
☐ React.js 공부하기	[삭제]
☐ REDUX 공부하기	[삭제]

실습파일 : 10_todo_app > src > App.jsx

SPA 만들기

☑ React.js App 개선 하기

- 현재 앱의 동작 과정을 확인해 보자(①~③ 순서대로의 변화)
- TodoInput에서 값이 입력될 때마다 TodoList가 렌더링됨
- List에 변화가 없음에도 쓸데없는 렌더링이 수행됨



- ① TodoInput에서 입력 발생
- ② App에 있는 input state 변경
- ③ TodoList의 rendering 실시

SPA 만들기

☑ React.js App 개선 하기

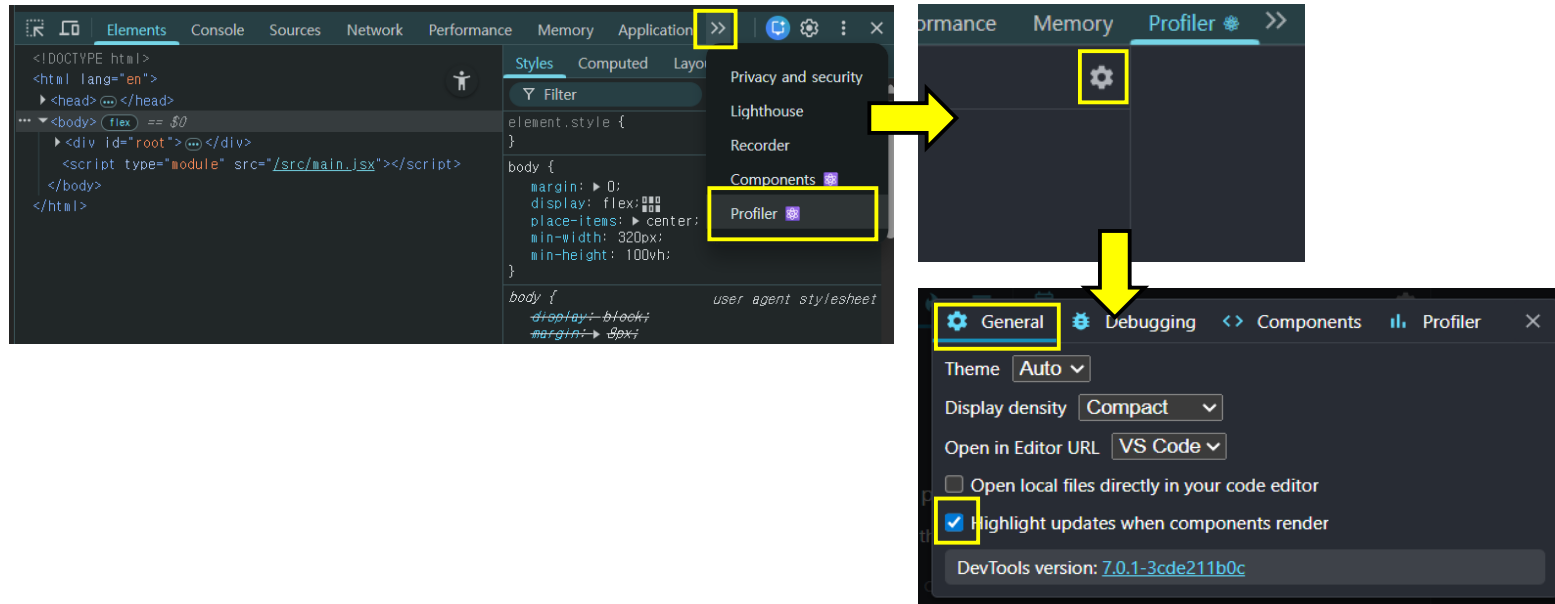
- 이때 useMemo()를 사용하여 불필요한 rendering을 줄일 수 있음

기존	Memo
App.jsx에서 state변화 시 re-rendering 될 때 하위 컴포넌트들이 다시 rendering	App.jsx에서 state 변화 시 re-rendering될 때 하위 컴포넌트 들은 자신이 지정한 state 변화에만 rendering

SPA 만들기

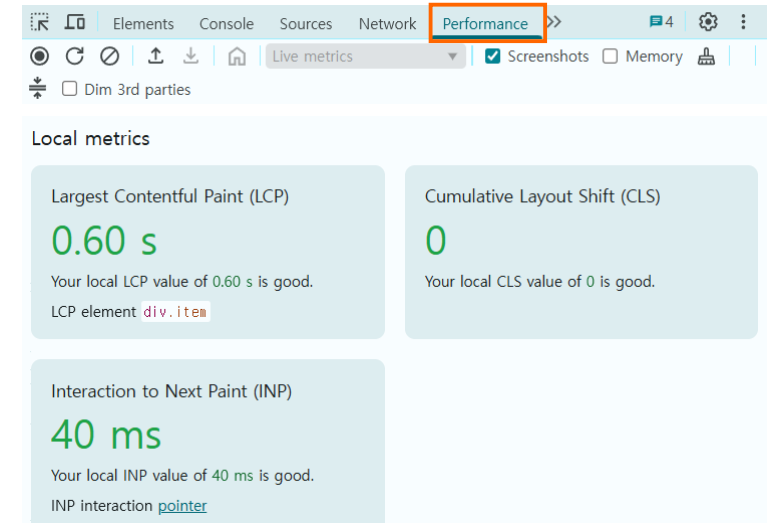
✓ React.js App 개선 하기

- 먼저 성능 비교를 위해 성능측정 도구를 설치해 보자
- Chrome에서 React Developer Tools 검색 후 추가 *설치 후 chrome 재시작
- 프로젝트 파일 열기 *React가 로드된 페이지에서만 확인 가능
- Profiler > 톱니바퀴 > General > Highlight updates... 를 check



추천 4.0 ★ (평점 1.6천개) < 공유

확장 프로그램 개발자 도구 4,000,000 사용자



SPA 만들기

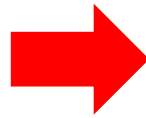
☑ React.js App 개선하기

- useMemo() 사용 후 호출 횟수와 성능을 확인해 보자
- 할 일 입력 -> 등록 -> 체크까지의 rendering log를 체크해 보자

Console useMemo() 사용 전

```
60 rendering
  todo app 만들기 insert!
  ▶ {id: 3, text: 'todo app 만들기', done: false}
4 rendering
4 rendering
```

실습파일 : 10_todo_app > src > comp > List.jsx



Console useMemo() 사용 후

```
todo app 만들기 insert!
  ▶ {id: 3, text: 'todo app 만들기', done: false}
rendering
rendering
```

실습파일 : 11_memo > src > comp > list.jsx

SPA 만들기



React.js App 개선 하기

- 소스에서의 변경점도 확인해 보자
- useMemo(()=>{},[]) 사용법도 확인해 보자

Console useMemo() 사용

```
const list = todos.map(item=>{
  // 입력, 체크 등 UI 에 변화가 생길 때마다 계속재 호출
  console.log('rendering');
  return(
    <Item key={item.id}
      done={item.done}
      done_yn={item.done === true ? "done" : "yet"}
      onToggle={()=>{onToggle(item.id)}}
      onRemove={()=>{onRemove(item.id)}}
    >{item.text}</Item>
  );
});
```

실습파일 : 10_todo_app > src > comp > List.jsx

Console useMemo() 사용 후

```
const list = useMemo(()=>{
  console.log('rendering');
  return(
    todos.map(item =>{
      <Item key={item.id}
        done={item.done}
        done_yn={item.done === true ?
          "done" : "yet"}
        onToggle={()=>{onToggle(item.id)}}
        onRemove={()=>{onRemove(item.id)}}
      >{item.text}</Item>
    )
  );
}, [todos]);
```

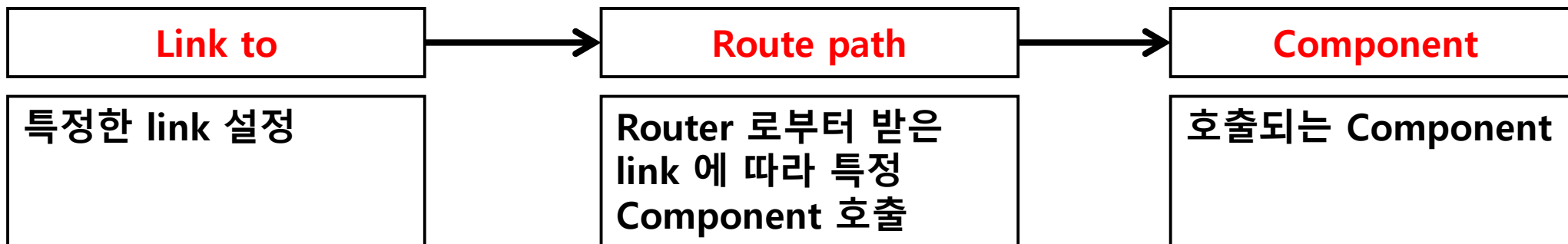
실습파일 : 11_memo > src > comp > List.jsx

Router

☑ Router 개념

- React.js는 SPA(Single Page App)을 위해 만들어진 Framework
- 페이지 이동을 지원하지 않음
- 페이지 이동을 위해서 React Router라는 library가 필요

```
npm install react-router-dom //npm 명령어를 활용한 react-router-dom 설치
```



Router

☑ router 기능을 활용해 보자

- 특정 링크(1)를 클릭 하면, 링크에 연결한 Component(2) 로 이동하게 된다.

```
<BrowserRouter>
  <div>
    <ul>
      <li><Link to="/">Home</Link></li>
      <li><Link to="/first">First</Link></li>
      <li><Link to="/second">Second</Link></li>
      <li><Link to="/topics">Topic</Link></li>
    </ul>
  </div>
  <Routes>
    <Route path="/" element={<Home/>}/>
    <Route path="/first" element={<First/>}/>
    <Route path="/second" element={<Second/>}/>
    <Route path="/topics/*" element={<Topics/>}/>
  </Routes>
</BrowserRouter>
```

- [Home](#)
- [First](#)
- [Second](#)
- [Topic](#)

Topics

- [Component](#)
- [Props vs State](#)

Requested topic ID: props_v_state

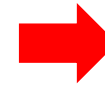
실습파일 : 12_router > src > App.jsx

Summary

☑ SPA 앱 구현

- SPA : Single Page Application
- 여러 Component 조합을 통해 SPA 를 생성
- useMemo() 기능을 통해 불필요한 rendering 줄일 수 있음

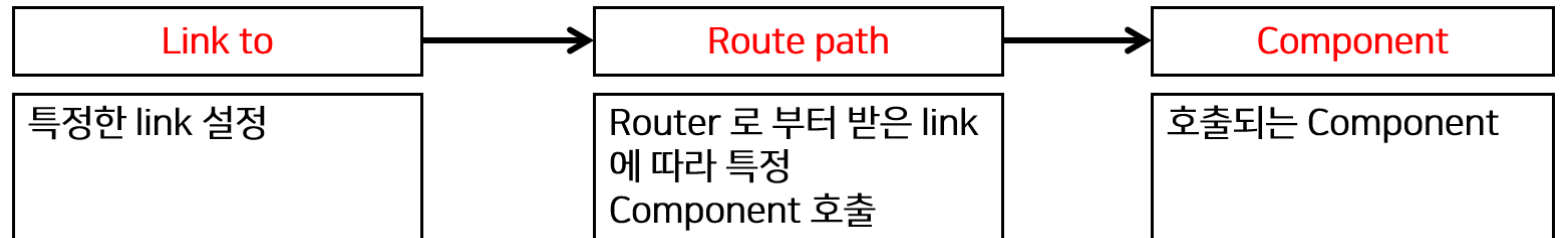
```
60 rendering
    todo app 만들기 insert!
    ▶ {id: 3, text: 'todo app 만들기', done: false}
4 rendering
4 rendering
```



```
todo app 만들기 insert!
▶ {id: 3, text: 'todo app 만들기', done: false}
rendering
rendering
```

☑ Router

- Router는 앱을 여러 페이지 형태로 구성 가능
- 외부 라이브러리를 설치 후 구현 가능
- Link를 통해 Router로 연결된 Component로 이동하는 형태



Chapter Summary

React.js Component

- React.js는 Component를 조합하여 U.I. 를 구성
- Component는 render() 함수를 통해 UI 를 rendering

Props vs state

- 둘 다 UI rendering에 영향을 주는 속성
- Props : 부모 Component로부터 받은 값
- State : Component 자체로 생성한 값

useEffect()

- Component 특정 생성 주기, 특정 상태(state)값 변화에 코드 수행이 가능

useMemo()

- 불필요한 rendering을 막아줌